08_ae_a

Section 6: Analytics Engineering Interview

This section covers the analytics-engineering layer that everything else in Helios stands on: the dbt project that turns raw GA4 events into governed marts, the semantic layer that is the single source of truth for every metric, and the dimensional modeling decisions (grain, layering, star schema, sessionization) that make the numbers correct and cheap to query. Throughout, I demonstrate each concept in general first and then ground it in the Helios dbt project, `models/semantic/semantic_models.yml` (28 metrics, 16 dimensions), and the `fct_*` / `dim_*` marts.

dbt

Q1. Walk me through how you structured a dbt project and chose materializations for each layer.

Ideal answer. I structure dbt as a strict one-directional DAG — sources to staging to intermediate to marts to a semantic layer — and pick the materialization per layer by trading off freshness, storage, and rebuild cost. Staging is `view` (thin rename/cast, no storage, always fresh); reusable business logic in intermediate is `ephemeral` so it inlines as CTEs and doesn't proliferate objects; the core day-grain facts are `incremental` so I only reprocess new partitions; small finance/growth facts are `table` because a full rebuild is cheap; and the semantic layer is `view` so it reflects marts instantly. The governing principle is: materialize where reuse or query cost justifies storage, otherwise stay a view. **Why Helios demonstrates it.** Helios' `dbt_project.yml` sets `staging: {+materialized: view}`, `intermediate: {+materialized: ephemeral}`, `marts.core: incremental`, `finance/growth: table`, `semantic: view` — exactly this layered policy, defaulted at the directory level so individual models stay clean. **Follow-ups.** When would you promote an ephemeral intermediate model to a table? / Why not materialize staging as tables to speed downstream reads? / What breaks if the semantic layer were a table instead of a view?

Q2. Explain incremental models in dbt and when you'd use one.

Ideal answer. An incremental model only transforms and appends/replaces new data on each run instead of rebuilding the whole table, gated by `is_incremental()` and a predicate that limits the source scan to recent rows. You use it when the table is large and append-mostly and a full

rebuild is too slow or too expensive — typical of event/fact tables. The key design choices are the incremental *strategy* (how new rows are reconciled with existing ones), the unique key (for merge), and a lookback window wide enough to absorb late-arriving data. The cost saving is real but it introduces state, so I always keep `--full-refresh` working as the safe reset. **Why Helios demonstrates it.** Helios' core funnel facts (`fct_funnel` , `fct_daily_funnel`) are `incremental` , partitioned by `event_date` , with an `is_incremental()` predicate `where event_date ≥ date_sub(_dbt_max_partition, interval 3 day)` so a freshly landed `events_YYYYMMDD` shard is processed without re-reading ~90 days of history. **Follow-ups.** How do you pick the lookback window? / What happens on the very first run of an incremental model? / How do you detect and recover from incremental drift?

Q3. Compare the `insert_overwrite` and `merge` incremental strategies on BigQuery. Which did you choose and why?

Ideal answer. `merge` matches new rows to existing rows on a unique key and updates/inserts row-by-row — flexible but requires a reliable surrogate key and pays a join cost. `insert_overwrite` atomically replaces *entire partitions* that appear in the new batch, which is cheaper on a date-partitioned table and inherently idempotent: reprocessing a corrected shard simply overwrites that day's partition with no dedup logic. I chose `insert_overwrite` partitioned by `event_date` because the GA4 export is date-sharded, so "the unit of correction is a day" maps perfectly onto "the unit of overwrite is a partition." That guarantees byte-identical output on re-runs without ever needing surrogate-key dedup. **Why Helios demonstrates it.**

```
{{ config(materialized='incremental',
   incremental_strategy='insert_overwrite',
   partition_by={'field':'event_date','data_type':'date','granularity':'day'}) }}
```

Helios uses exactly this config on core marts; a late or corrected GA4 shard reprocesses cleanly because the whole `event_date` partition is replaced. **Follow-ups.** What's the risk if a late event lands outside your lookback window with `insert_overwrite` ? / When would `merge` be unavoidable here? / How does `insert_overwrite` interact with BigQuery's `require_partition_filter` ?

Q4. How do partitioning and clustering work on BigQuery, and how did you choose your partition and cluster keys?

Ideal answer. Partitioning physically segments a table (here by `DATE`) so a query with a date predicate prunes whole partitions and never reads them — coarse but huge savings. Clustering sorts data within each partition by chosen columns so block-level pruning further drops bytes on filters and joins on those columns — a fine-grained second filter. I partition on the column every query filters and cluster on the highest-cardinality, most-frequently-filtered dimensions. The two

compose: partition pruning first, then cluster pruning. **Why Helios demonstrates it.** Every Helios fact partitions by `event_date` (the diagnosis window is always a date range, so a 14-day diagnosis scans ~14 of ~90 partitions) and clusters by `device_category, channel_group` — the two dimensions the Decompose agent slices on most when separating mix effect from rate effect. I also set `require_partition_filter = true` so a query missing a date predicate errors instead of full-scanning. **Follow-ups.** Why not partition by `event_date` *and* cluster by it too? / What's the cardinality limit on a cluster key before it stops helping? / How would you partition if queries filtered on `channel_group` more than date?

Q5. What are dbt macros, and how have you used them to enforce consistency?

Ideal answer. A macro is a reusable Jinja function that emits SQL, letting me define a transformation once and call it everywhere — the DRY principle applied to SQL. The high-value use is encoding business logic that must be identical across models: extracting a typed event parameter, building a key, or classifying a dimension. Centralizing it means a rule change is a one-file edit and there's no risk of two models drifting apart. The tradeoff is that over-macro'd SQL becomes hard to read, so I reserve macros for logic that is genuinely repeated or genuinely error-prone. **Why Helios demonstrates it.** Helios has four macros — `get_event_param(key,type)` (one canonical UNNEST extraction), `sessionize()` (builds the `(user_pseudo_id, ga_session_id)` key), `channel_group_case()` (the single source of truth for all 10-channel classification), and a custom `revenue_reconciles` test. Because `channel_group_case()` lives in exactly one macro, an attribution-rule change can never silently diverge between `int_ga4__sessionized` and the marts. **Follow-ups.** How do you unit-test a macro? / When does a macro belong in a package instead of the project? / How do you document a macro so a reviewer trusts it?

Q6. How do you use dbt packages, and which have you relied on?

Ideal answer. Packages are versioned dependencies declared in `packages.yml` and pulled via `dbt deps`; they ship reusable macros and generic tests so I don't reinvent common utilities. The discipline is pinning a version range so a transitive update can't silently change my output, and preferring battle-tested packages over hand-rolled SQL for things like surrogate-key generation, date spines, and range/expression tests. I keep the dependency surface small — every package is code I'm implicitly trusting in my build. **Why Helios demonstrates it.** Helios pins `dbt-labs/dbt_utils` (`≥1.1.0, <2.0.0`) and uses it for generic tests like `dbt_utils.accepted_range` on `revenue` (min 0) — keeping schema tests declarative while the project's own logic stays in custom macros and a custom `revenue_reconciles` generic test. **Follow-ups.** Why pin a version range rather than a single version? / Would you use `dbt_utils.generate_surrogate_key` or hand-roll `TO_HEX(MD5( … ))`, and why? / What's the risk of relying on a package macro that changes behavior across versions?

Q7. What's the difference between `dbt run`, `dbt test`, and `dbt build`, and which do you run in CI?

Ideal answer. `dbt run` executes model SQL (materializes models); `dbt test` runs data/schema tests against already-built models; `dbt build` interleaves them in DAG order — it runs each model and then immediately runs that model's tests before proceeding, and also handles seeds and snapshots. `build` is the right default because it stops the pipeline at the first model whose tests fail instead of building a whole broken downstream graph on bad data. In CI I run `dbt build` so a contract violation blocks the merge; ad hoc I might `dbt run --select` a subgraph while iterating. **Why Helios demonstrates it.** Helios' production refresh and CI both use `dbt build` (`--select state:modified+` in CI), so a reconciliation or schema test failing on `fct_orders` halts the build and blocks the agent run rather than letting the diagnosis pipeline reason over silently-wrong marts. **Follow-ups.** Why does `build` catch bugs `run` then `test` would miss? / How does `build` order tests relative to downstream models? / When would you deliberately `run` without `test`?

Q8. Explain Slim CI in dbt and how you implemented it.

Ideal answer. Slim CI builds and tests only the models that changed plus their downstream children, instead of the whole project, by comparing the PR's code against a stored production manifest (`state:modified+`) and deferring unbuilt upstream refs to prod (`--defer --state`). This cuts CI runtime and warehouse cost dramatically on large projects while still proving that a change and everything it touches is valid. The prerequisite is publishing a fresh prod `manifest.json` artifact that CI can diff against. **Why Helios demonstrates it.**

```
- run: dbt build --select state:modified+ --defer --state ./prod-manifest
- run: dbt test  --select state:modified+
- run: python eval/run_benchmark.py   # root-cause accuracy gate ≥85%
```

Helios' GitHub Actions CI uses `state:modified+ --defer` to keep PR builds under the fixed BigQuery byte budget, then layers the labeled eval gate on top so a change that drops root-cause accuracy below 85% (vs the <=45% naive baseline) fails the PR. **Follow-ups.** How do you keep the prod manifest fresh? / What does `--defer` actually do to unbuilt upstream refs? / What's a change Slim CI would *miss* that a full build would catch?

Q9. How do you guarantee a dbt model is idempotent, and why does it matter for an autonomous system?

Ideal answer. Idempotent means re-running over the same input window yields byte-identical output — no duplicates, no drift, no dependence on run count or wall-clock. I get it by making each run a function of the data window only: deterministic keys, partition-atomic overwrite (so a

partition is fully replaced, never appended-to twice), and no `current_timestamp`-style nondeterminism baked into stored columns. It matters for an autonomous system because the scheduler will re-run on retries and late shards; without idempotency a transient failure quietly corrupts history and every downstream finding inherits the error. **Why Helios demonstrates it.** Helios pairs `partition_by event_date` with `insert_overwrite` so reprocessing a day overwrites exactly that partition, and the `revenue_reconciles` test asserts `sum(revenue)` from `fct_orders` equals the raw source total to the cent — any non-idempotent drift fails the build and blocks the agent run. **Follow-ups.** How would `merge` without a stable key break idempotency? / Where might a non-deterministic function sneak into a "deterministic" model? / How do you test idempotency in CI?

Semantic Layers

Q10. What is a semantic layer and why does an analytics org need one?

Ideal answer. A semantic layer is a governed, central definition of metrics and dimensions — names, formulas, grains, and allowed slices — that sits between physical tables and every consumer (BI tools, notebooks, APIs, agents). Its job is to make "revenue" or "conversion rate" mean exactly one thing everywhere, so two dashboards can't compute it two ways. Organizationally it ends the "whose number is right?" debate, lets analysts self-serve without re-deriving SQL, and gives one place to fix a definition when the underlying schema changes. The cost is upfront modeling discipline and the governance to keep ad hoc SQL from bypassing it. **Why Helios demonstrates it.** Helios' semantic layer is `models/semantic/semantic_models.yml` — 28 metrics and 16 dimensions defined once — and it's not advisory: `semantic-mcp` is the *only* path to SQL, so no consumer (including the LLM) can compute a metric off-definition. **Follow-ups.** How is a semantic layer different from a well-modeled mart? / Who should own metric definitions — analytics engineering or the business? / What's the failure mode when a semantic layer is optional rather than enforced?

Q11. Walk me through the schema of your metric and dimension registry.

Ideal answer. Each metric is a typed YAML record: a canonical snake_case `name`, `label`, `description`, a `type` (count / sum / ratio / derived), the `entity` and physical `grain` it resolves against, the aggregation (`agg`) and additive `sql` for count/sum, `numerator`/`denominator` for ratios, an `expr` over other metrics for derived, governed `filters`, a `format`, a `dimensions` whitelist, an `owner`, and a semver `version`. Dimensions are simpler: `name`, `type` (categorical/temporal/boolean), `entity`, the resolving `sql`, and `format`. The whitelist and grain fields are the load-bearing ones — they constrain *what can be sliced by what*, which is the difference between a definition store and a true governance layer. **Why Helios demonstrates it.** Helios' registry encodes all of this; for example

`session_conversion_rate` is `type: ratio`, `grain: fct_funnel`, `numerator: purchasing_sessions`, `denominator: sessions`, with a `dimensions` whitelist of the 14 session dimensions and `version: 2.0.0`. **Follow-ups.** Why carry `entity` and `grain` separately? / What does the `dimensions` whitelist buy you over allowing any dimension? / Why semver each metric individually?

Q12. How does a registry of definitions actually prevent the LLM from hallucinating columns?

Ideal answer. Because the consumer never writes SQL — it passes *names*, and a deterministic resolver composes the SQL from the registry. If a caller references a metric or dimension that isn't registered, the lookup raises before any SQL exists; an invented `conversion_pct` can't become a query, it becomes a hard error. Physical column names live exclusively in the registry's `sql` fields owned by analytics engineers, so the model literally has no surface on which to emit a column. This converts "0 hallucinated columns" from a hope into a structural property of the system. **Why Helios demonstrates it.** Helios' `semantic-mcp.build_query(metric, dims, filters, window)` does `REGISTRY.metrics[metric]` (KeyError = hard fail) and rejects any dim not in that metric's whitelist before composing SQL — so the LLM passing a bad name is caught at the boundary, then the Critic agent flags it and the model retries with a real metric. **Why this is grounding rule G5.** An unknown name is a hard error, never a fallback to free SQL — that's the anti-hallucination guarantee in one rule. **Follow-ups.** What stops the LLM from calling `warehouse-mcp.run_query` directly to bypass the registry? / How do you validate the registry itself is internally consistent? / Where does reconciliation fit on top of this?

Q13. What is "governed SQL," and how is governance enforced beyond just a YAML file?

Ideal answer. Governed SQL means every query a consumer runs is composed from versioned, owned, tested definitions rather than hand-authored — the YAML is the contract and a deterministic resolver is the only author. Enforcement is layered: the resolver rejects unknown names; CI compiles the registry and fails on dangling references or schema drift; and — critically — *credentials* are scoped so the path can't be bypassed. The deepest enforcement isn't software politeness, it's IAM: the service account that backs the semantic path can only read the governed views, not raw tables. **Why Helios demonstrates it.** Helios separates the `helios_semantic` dataset from `helios_marts`, and `sa-helios-semantic` (backing `semantic-mcp`) is granted read on `helios_semantic` *only* — so the LLM physically cannot reach raw or even mart tables to hand-author SQL. CI compiles the registry's referential integrity on every PR. **Follow-ups.** What's the threat model if all servers shared one service account? / How do governed `filters` (row-level governance) work in your registry? / How do you handle a legitimate one-off query the registry can't express?

Q14. What is dbt MetricFlow / the dbt Semantic Layer, and how does your registry map to it?

Ideal answer. MetricFlow is dbt's semantic-layer engine: you define *semantic models* over facts (with entities, dimensions, and measures), then define metrics on top (`simple` , `ratio` , `derived` , `cumulative`), and MetricFlow compiles and queries them so consumers ask for metrics-by-dimensions without writing joins or `GROUP BY` . My registry maps onto it almost 1:1: each `entity` / `grain` becomes a semantic model over a fact, additive `agg: sum|count_distinct` metrics become MetricFlow measures, `type: ratio` becomes a ratio metric, `type: derived` becomes a derived metric, and dimension `sql` becomes semantic-model dimensions. **Why Helios demonstrates it.** Helios' YAML maps 1:1 to MetricFlow — `entity: session` / `grain: fct_funnel` is a semantic model over `fct_funnel` , `aov` is a ratio metric (`gross_revenue` / `transactions`), `revenue_per_session` is a derived metric (`SAFE_DIVIDE({revenue},{sessions})`) — but I ship a thin custom resolver so it runs identically with or without a dbt Cloud Semantic Layer endpoint; the YAML is the single source of truth either way. **Follow-ups.** Why ship a custom resolver instead of depending on dbt Cloud? / How does MetricFlow prevent fan-out/fan-trap double counting across grains? / What does MetricFlow give you that a wide pre-aggregated table doesn't?

Q15. Why is "single source of truth" so important, and what concretely happens when a definition changes?

Ideal answer. A single source of truth means a metric's formula and the dimension classification logic each live in exactly one place, so consistency is structural rather than maintained by vigilance. When the definition changes, you edit one file, bump the version, and every consumer picks up the new behavior on the next build — and prior outputs remain interpretable because they're tagged with the version that produced them. The anti-pattern is the same CASE statement copy-pasted across five models: a change becomes a five-place edit where one place inevitably gets missed and silently diverges. **Why Helios demonstrates it.** Helios enforces two single sources of truth: every metric/dimension lives only in `semantic_models.yml` , and *all* channel-grouping logic lives only in the `channel_group_case()` macro. Definitions carry `owner` + semver; a breaking formula change bumps the major version and is recorded in the Memory store so old diagnoses stay interpretable, and the Critic checks a finding's cited metric version against the run's registry version. **Follow-ups.** How do you version a metric without breaking historical dashboards? / Where does channel logic belong — the macro or the registry — and why both? / How do you migrate consumers off a deprecated metric?

Q16. How do you test and validate a semantic layer itself, separate from the data?

Ideal answer. Two distinct kinds of validation. First, *structural* validation at compile time: every `numerator` / `denominator` / `expr` token must resolve to a registered metric, and every dimension in a whitelist must be a registered dimension on a compatible entity — dangling references fail the build. Second, *value* validation against reality: reconcile the composed metric's total against a

canonical raw total, because a perfectly-consistent registry can still encode a wrong formula. The first proves the layer is coherent; the second proves it's correct. **Why Helios demonstrates it.** Helios compiles `semantic_models.yml` in CI with referential-integrity checks (dangling token = hard fail), and on top of that `warehouse-mcp.reconcile` asserts mart/semantic totals agree with raw canonical totals within <=0.5%; the `revenue_reconciles` test pins `revenue` to the source to the cent. **Follow-ups.** How do you unit-test a `derived` metric's algebra? / What reconciliation tolerance is acceptable and why not always zero? / How do you catch a metric that's self-consistent but semantically wrong?

Q17. A stakeholder asks for a metric your semantic layer doesn't have. Walk me through adding it.

Ideal answer. I add it as a definition, not as ad hoc SQL: a new YAML record with name, type, grain, the additive `sql` or numerator/denominator/expr, a dimensions whitelist, an owner, and version `1.0.0`. CI then compiles the registry, runs referential-integrity checks, and — if it's a new fact measure — I add a reconciliation test. Code review confirms the grain is declared and tested and no physical column was hardcoded into a mart. Only after that does the metric become callable by name; this keeps the "0 hallucinated, 100% governed" guarantee intact instead of letting one urgent request open a hole. **Why Helios demonstrates it.** Helios' CLAUDE.md rule is explicit: "add new metrics by editing the registry YAML, not by hand-writing SQL." A new metric flows through the same governance gate (compile, referential integrity, reconciliation, review) as every existing one of the 28. **Follow-ups.** How do you decide the new metric's grain? / What if the metric needs a dimension that doesn't exist yet? / How do you avoid metric sprawl — 200 near-duplicate definitions?

Modeling

Q18. Explain the staging / intermediate / marts layering and what belongs in each.

Ideal answer. Staging is a 1:1, lightly-cleaned mirror of each source — rename to snake_case, cast types, expose keys, no business logic — so downstream models never touch raw shapes. Intermediate holds reusable business logic that more than one mart needs (sessionization, flagging, channel classification) and exists to avoid duplicating that logic into every mart. Marts are the consumption layer: grain-explicit facts and conformed dimensions modeled for the questions analysts actually ask. The discipline is that logic flows in one direction and each layer has a single responsibility — staging cleans, intermediate computes shared logic, marts shape for consumption. **Why Helios demonstrates it.** Helios runs `src_ga4` -> `stg_ga4__events` / `stg_ga4__event_params` (typed views, one UNNEST of `event_params` done once) -> `int_ga4__sessionized` / `int_ga4__funnel_steps` (session key, channel group, `reached_*` flags) -> marts (`fct_funnel` , `fct_daily_funnel` , `fct_orders` , `dim_*`). The

semantic layer is the fifth, governance layer on top. **Follow-ups.** Why is `event_params` unnested once in staging rather than per-mart? / What's the smell when business logic leaks into staging? / Why have intermediate models at all instead of long mart CTEs?

Q19. What is a star schema, and why prefer it over a fully normalized (snowflake) model for analytics?

Ideal answer. A star schema is central fact tables (the measurable events) surrounded by denormalized dimension tables (the descriptive context), joined on surrogate keys. It's preferred for analytics because the wide, denormalized dimensions minimize joins at query time and are intuitive to slice, which matters far more for read-heavy BI than the storage savings normalization gives you. A snowflake further normalizes dimensions into sub-dimensions, saving space but adding joins and cognitive load — rarely worth it on a warehouse where storage is cheap and scans are the cost. So I model a star: conform shared dimensions, keep facts at a clean grain, and denormalize dims. **Why Helios demonstrates it.** Helios' marts are a star: `fct_funnel` / `fct_sessions` (session grain) and `fct_orders` (transaction grain) at the center, with conformed `dim_users`, `dim_items`, `dim_channels`, `dim_date` around them, so the Decompose agent can pivot any metric across `device_category`, `channel_group`, `country` without runtime join gymnastics. **Follow-ups.** When would you accept a snowflaked dimension? / What makes a dimension "conformed," and why does it matter for the decomposition? / How do slowly-changing dimensions fit the star?

Q20. What is grain, why must you declare it first, and how do you defend it?

Ideal answer. Grain is the precise meaning of one row in a table — "one row per X." You declare it before writing any SQL because every column, key, and aggregation depends on it; a model without an agreed grain is a model whose numbers no one can trust. You defend it with a uniqueness test on the grain key: if `transaction_id` is supposed to be unique and isn't, the fact is double-counting and the test fails the build. Mixed or ambiguous grain is the single most common cause of fan-out double counting, so grain is the first thing I write down and the first thing I test. **Why Helios demonstrates it.** Every Helios fact states its grain explicitly — `fct_funnel` is one row per session (PK `session_key`), `fct_orders` is one row per `transaction_id`, `fct_order_items` is one row per transaction-x-item line — and each grain key carries `unique` + `not_null` tests so a grain violation fails CI. **Follow-ups.** How does wrong grain cause a fan-out, concretely? / Why is `count(distinct)` your friend when grain is uncertain? / How do you reconcile two facts at different grains in one query?

Q21. What are surrogate keys, why use them, and how did you build yours?

Ideal answer. A surrogate key is a single synthetic key that stands in for a natural or composite business key, usually a hash of the columns that define the grain. You use them because they give

every row one stable join key, hide composite-key complexity from consumers, and stay stable across runs if generated deterministically. The rule is determinism: hash the *exact* grain columns the same way everywhere so the key is reproducible and idempotent. I avoid keys that aren't stable across runs and I always hash the full natural key, not a subset. **Why Helios demonstrates it.** Helios' canonical session key is `session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))` — one deterministic expression used everywhere (never `FARM_FINGERPRINT`, never `COUNT(*)`), so `sessions = COUNT(DISTINCT session_key)` is reproducible run to run. `dim_channels` similarly uses `TO_HEX(MD5(channel_group))` as `channel_key`. **Follow-ups.** Why MD5-hex rather than `FARM_FINGERPRINT` or a row_number? / What happens to the key if one grain column is null? / How does a deterministic surrogate key support `insert_overwrite` idempotency?

Q22. Why build wide (denormalized) fact tables, and what's the tradeoff?

Ideal answer. A wide fact carries the full descriptive dimension set on the fact row itself (the Kimball wide-fact / one-big-table pattern) so the consumption layer can slice any metric by any dimension without runtime joins. The benefit is query simplicity and speed, and on a columnar warehouse the extra columns are nearly free because unselected columns aren't scanned. The tradeoff is storage and the need to keep denormalized attributes consistent with their dimension source — which I manage by deriving those attributes once upstream rather than per-fact. For agent- or BI-driven slicing across many dimensions, wide facts are usually the right call. **Why Helios demonstrates it.** Helios' architectural ruling makes `fct_funnel` / `fct_sessions` wide — carrying the full session dimension set (`device_category`, `operating_system`, `browser`, `country`, `region`, `channel_group`, `source`, `medium`, `campaign`, `landing_page`, `is_new_user`, `ga_session_number`) alongside the `reached_*` flags and `session_revenue` — so the semantic layer slices any metric by any of those without a runtime join, which the diagnosis tree needs to drill every dimension. **Follow-ups.** How do you keep a wide fact's denormalized attributes from drifting from the dimension? / When is the join cost worth keeping the fact narrow? / How does columnar storage change the wide-vs-narrow calculus?

Q23. Sessionization isn't given in GA4 — how did you reconstruct sessions, and what are the pitfalls?

Ideal answer. GA4's BigQuery export is event-grained — there is no session row — so a session is reconstructed by grouping events that share a session identifier. The identifier lives inside the `event_params` array, so you UNNEST to extract `ga_session_id`, and critically you key on the *composite* `(user_pseudo_id, ga_session_id)` because `ga_session_id` is only unique within a user, never globally. The pitfalls are: keying on `ga_session_id` alone (cross-user collisions), forgetting that identity is cookie-grain since `user_id` is almost always null, and deriving session attributes from the wrong scope. I collapse events to one session row in an intermediate model

and hash the composite key into a surrogate. **Why Helios demonstrates it.** Helios' `int_ga4__sessionized` collapses events to one row per `(user_pseudo_id, ga_session_id)`, hashes it into `session_key`, and derives session-scoped source/medium/channel and `reached_*` flags there — so downstream marts never re-implement UNNEST or re-derive the session. **Follow-ups.** Why is `(user_pseudo_id, ga_session_id)` the key rather than `ga_session_id` alone? / How do you handle a session that spans midnight / two date shards? / What's the identity limitation when `user_id` is null and how does it bias user-level metrics?

Q24. The GA4 `traffic_source` field is a trap — explain it and how your model handles it.

Ideal answer. The event-level `traffic_source` struct in the GA4 export is *user first-touch* attribution — stamped from the user's very first acquisition and copied onto every later event — not the source of the session that fired the event. Using it for session-level channel analysis systematically over-credits acquisition channels and under-credits re-engagement, and it can manufacture exactly the Simpson's-paradox confounds a diagnosis engine is trying to detect. The fix is to derive session-scoped source/medium from the session's own `event_params` (first non-null value ordered by timestamp) and use `traffic_source` only as a documented fallback when session params are entirely null. **Why Helios demonstrates it.** Helios derives `session_source` / `session_medium` in `int_ga4__sessionized` from `event_params` (with `(direct)` / `(none)` normalization), feeds them into `channel_group_case()`, and only falls back to first-touch `traffic_source` when session scope is null — preventing first-touch leakage from corrupting the channel-level decomposition. **Follow-ups.** How does first-touch leakage create a fake mix-shift? / When is first-touch actually the right attribution (e.g., `dim_users.first_touch_channel`)? / How do you reconcile your derived channel split against GA4's own UI?

Q25. How do you compute a session-level conversion rate so it survives Simpson's paradox when aggregated?

Ideal answer. Materialize monotonic per-session boolean step flags, then compute every rate as `SUM(numerator) / SUM(denominator)` *after* grouping — never as the average of per-segment ratios. Averaging ratios weights segments equally regardless of size, which is precisely how Simpson's paradox produces a topline that moves opposite to every segment. Re-aggregating the raw numerator and denominator preserves the correct volume weighting at any grain. The flags must be max-downstream (reached step X = reached X or any later step) so they're monotonic and step rates can never exceed 1. **Why Helios demonstrates it.**

```
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNT(DISTINCT session_key)) AS session_conversion_rate
```

Helios materializes `reached_*` max-downstream flags in `int_ga4__funnel_steps`, computes rates as SUM/SUM in `fct_daily_funnel`, and `stats-mcp.decompose_change` then splits any topline rate move into mix / rate / interaction — turning the re-aggregation discipline into an explicit, deterministic Simpson's-paradox defense. **Follow-ups.** Show me numerically how averaging ratios flips the topline. / Why must the flags be max-downstream rather than literal step-fired? / How does the mix/rate/interaction split formally dissolve the paradox?